

Comparator — 정렬 기준을 정의하는 함수형 인터페이스

코딩테스트 정렬 문제의 핵심. 원하는 기준으로 정렬할 때 반드시 필요하다.

계층 위치: 함수형 인터페이스 (Collection 계층과 독립) — `list.sort(comparator)`로 연결

Comparable vs Comparator

	Comparable	Comparator
위치	객체 내부에 구현	객체 외부에서 정의
메서드	<code>compareTo(T o)</code>	<code>compare(T a, T b)</code>
의미	"나는 기본적으로 이렇게 정렬된다"	"이 기준으로 정렬해라"
예시	<code>String</code> 은 기본 사전순	길이순, 역순 등 커스텀

코딩테스트에서 **Comparator**를 훨씬 많이 쓰는 이유: 문제마다 정렬 기준이 다르기 때문에, 매번 외부에서 기준을 주입하는 Comparator가 유연하다. Comparable은 클래스를 수정해야 하므로 코딩테스트에서는 비실용적.

핵심 메서드

메서드	설명	시간복잡도
<code>compare(a, b)</code>	$a < b \rightarrow$ 음수, $a == b \rightarrow 0$, $a > b \rightarrow$ 양수	$O(1)$
<code>Comparator.comparingInt(keyExtractor)</code>	int 키 기준 오름차순	$O(1)$
<code>Comparator.comparingLong(keyExtractor)</code>	long 키 기준 오름차순	$O(1)$
<code>Comparator.comparing(keyExtractor)</code>	Comparable 키 기준 오름차순	$O(1)$
<code>thenComparing(keyExtractor)</code>	1차 정렬 후 2차 정렬	$O(1)$
<code>reversed()</code>	정렬 순서 뒤집기	$O(1)$
람다 $(a, b) \rightarrow a - b$	오름차순 (가장 많이 쓰는 형태)	$O(1)$
람다 $(a, b) \rightarrow b - a$	내림차순	$O(1)$

패턴

코딩테스트 출제 빈도: 패턴1 기본 오름차순/내림차순 > 패턴2 문자열 이어붙이기 커스텀 정렬 > 패턴3 내림차순 + 인덱스 조건 탐색 > 패턴4 다단계 정렬

패턴 1: 기본 오름차순/내림차순

언제 쓰는가

- 단순 정렬이 필요할 때

실무에서는

- 검색 결과 정렬, 시간순 정렬

```
import java.util.*;

public class BasicSort {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(Arrays.asList(5, 2, 8, 1, 9));

        // 오름차순
        nums.sort((a, b) -> a - b);
        System.out.println(nums); // [1, 2, 5, 8, 9]

        // 내림차순
        nums.sort((a, b) -> b - a);
        System.out.println(nums); // [9, 8, 5, 2, 1]

        // comparingInt 사용 (더 명시적)
        nums.sort(Comparator.comparingInt(Integer::intValue));
        System.out.println(nums); // [1, 2, 5, 8, 9]

        // reversed()로 내림차순
        nums.sort(Comparator.comparingInt(Integer::intValue).reversed());
        System.out.println(nums); // [9, 8, 5, 2, 1]
    }
}
```

패턴 2: 문자열 이어붙이기 커스텀 정렬 — "가장 큰 수" 패턴

언제 쓰는가

- 기본 정렬 기준으로 해결할 수 없는 특수한 정렬
- 대표 문제: 숫자 배열을 이어붙여서 가장 큰 수 만들기

실무에서는

- 버전 번호 정렬, 자연스러운 파일명 정렬

```
import java.util.*;

public class LargestNumber {
    public static void main(String[] args) {
        // 숫자를 이어붙여 가장 큰 수 만들기: [6, 10, 2] -> "6210"
    }
}
```

```
String[] nums = {"6", "10", "2"};

// "610" vs "106" → "610"이 더 크므로 6이 10보다 앞
Arrays.sort(nums, (a, b) -> (b + a).compareTo(a + b));

StringBuilder sb = new StringBuilder();
for (String num : nums) {
    sb.append(num);
}

// 모두 0인 경우 처리
String result = sb.toString().startsWith("0") ? "0" :
sb.toString();
System.out.println(result); // 6210
}
```

패턴 3: 내림차순 + 인덱스 조건 탐색 — reversed()

언제 쓰는가

- 가장 큰 값부터 조건을 확인해야 할 때
- "최소 몇 개를 선택하면 조건을 만족하는가" 유형

실무에서는

- Top-N 추출, 랭킹 시스템

```
import java.util.*;

public class ReversedSearch {
    public static void main(String[] args) {
        // 로프 문제: 내림차순 정렬 후 i번째 로프의 값 * (i+1)의 최대값
        List<Integer> ropes = new ArrayList<>(Arrays.asList(2, 10, 8));

        ropes.sort(Comparator.reverseOrder()); // [10, 8, 2]

        int maxWeight = 0;
        for (int i = 0; i < ropes.size(); i++) {
            // i번째 로프까지 사용 → 가장 약한 로프 * 개수
            int weight = ropes.get(i) * (i + 1);
            maxWeight = Math.max(maxWeight, weight);
        }

        System.out.println("최대 중량: " + maxWeight); // 최대 중량: 16 (8 * 2)
    }
}
```

패턴 4: 다단계 정렬 — thenComparing

언제 쓰는가

- 1차 기준이 같을 때 2차 기준으로 정렬해야 할 때
- "점수 같으면 이름순" 같은 조건

실무에서는

- 게시물 정렬 (좋아요 → 최신순), 성적표 (총점 → 이름순)

```
import java.util.*;

public class MultiLevelSort {
    public static void main(String[] args) {
        // [이름, 점수] - 점수 내림차순, 같으면 이름 오름차순
        List<int[]> students = new ArrayList<>();
        students.add(new int[]{0, 85}); // 이름을 인덱스로 대체
        students.add(new int[]{1, 92});
        students.add(new int[]{2, 85});
        students.add(new int[]{3, 92});

        String[] names = {"Charlie", "Alice", "Bob", "David"};

        // 점수 내림차순 → 이름 오름차순
        students.sort((a, b) -> {
            if (a[1] != b[1]) return b[1] - a[1]; // 점수 내림차순
            return names[a[0]].compareTo(names[b[0]]); // 이름 오름차순
        });

        for (int[] s : students) {
            System.out.println(names[s[0]] + ": " + s[1]);
        }
        // Alice: 92
        // David: 92
        // Bob: 85
        // Charlie: 85

        // == thenComparing 활용 (객체 리스트일 때 더 깔끔) ==
        List<String> words = new ArrayList<>(Arrays.asList("banana",
            "fig", "apple", "kiwi", "date"));

        // 길이 오름차순 → 같은 길이면 사전순
        words.sort(Comparator.comparingInt(String::length)
            .thenComparing(Comparator.naturalOrder()));

        System.out.println(words); // [fig, date, kiwi, apple, banana]
    }
}
```

주의사항

sort()는 원본을 변경한다 (in-place)

```
List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 2));  
list.sort(Comparator.naturalOrder());  
// list 자체가 [1, 2, 3]으로 변경됨 - 새 리스트가 아님
```

comparingInt vs 람다 — 가독성 차이

```
// 동일한 동작, 스타일 차이  
list.sort((a, b) -> a - b); // 간결하지만 의도 불명확  
list.sort(Comparator.comparingInt(Integer::intValue)); // 명시적  
// 코딩테스트에서는 람다가 더 빠르게 쓸 수 있어서 선호됨
```

안정 정렬 (Stable Sort)

- Java의 `Arrays.sort()` (객체)와 `Collections.sort()`는 **TimSort** 사용 → 안정 정렬
- 같은 키 값을 가진 요소들의 **원래 순서가 유지됨**
- 다단계 정렬에서 중요: 2차 기준으로 먼저 정렬하고 1차 기준으로 다시 정렬해도 올바른 결과